

Product → Engineering Development Workflow

You describe the idea; the AI agent writes the spec, designs the screens, and writes the test cases. Two checkpoints run in an independent fresh session — one challenges the idea, one audits the documents — then the agent creates the pull request.

 Agent interviews you · 7 steps

 2 independent reviews

 Agent runs git + opens the PR

v4 · 2026-06-17

00 The mental model

Read this first — it is the one idea everything else hangs on.

This repository is the single place where a feature is born, challenged, specified, designed, tested, audited, and built. The docs *are* the spec, the `.pen` file *is* the design, and code is generated against both.

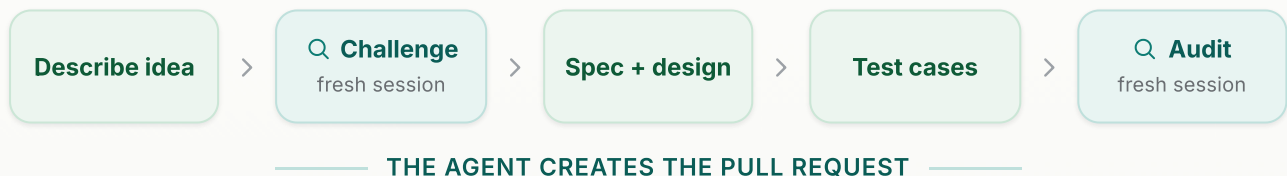
🛡️ THE GOLDEN RULE

Nothing reaches engineering that isn't traceable — every feature flows *epic* → *feature* → *PRD* → *acceptance criteria* → *test cases* → *design frame*, each carrying stable IDs the code and tests cite back. The author agent guarantees this and proves it's testable; **two independent review sessions** make sure the idea is worth building and the documents fit the existing system.

🔍 WHY A FRESH SESSION FOR THE TWO REVIEWS

The agent that interviewed you and wrote the spec is invested in its own work and blind to its own gaps. Opening a **new Claude Code session** for the **Challenge (02)** and the **Audit (06)** gives a reviewer that has *only* read the repo and the documents — not your conversation — so the critique is genuinely independent. **Independence is the point — don't skip it.**

🔗 PRODUCT — DESCRIBE & APPROVE · TEAL = INDEPENDENT FRESH-SESSION REVIEW



🔗 ENGINEERING — BUILD & SHIP



The whole loop. You describe and approve across the top; the two **teal** steps are independent fresh-session reviews. The agent creates the pull request; engineering owns the API contract + code and automates your test cases back to `main`.

Who does what

AREA	PRODUCT — YOU DESCRIBE & APPROVE	ENGINEERING — BUILDS
Specs	answer the interview; approve the FEATURE + PRD the agent writes	—
Design	approve the screens the agent designs in <code>brainstorm.pen</code>	—
Test cases	the agent generates & validates <code>TC-#</code> ; you approve coverage	implements them as automated tests
Independent review	run the Challenge (02) and Audit (06) sessions; act on the verdicts	—
Pull request	the agent creates the PR for your branch; you approve	reviews, completes, merges
API · data · code	<i>you never touch these</i>	API contract, legacy data-mapping, code

01 One-time setup

You do this once — an engineer can help in five minutes. After this you only ever talk to the agent.

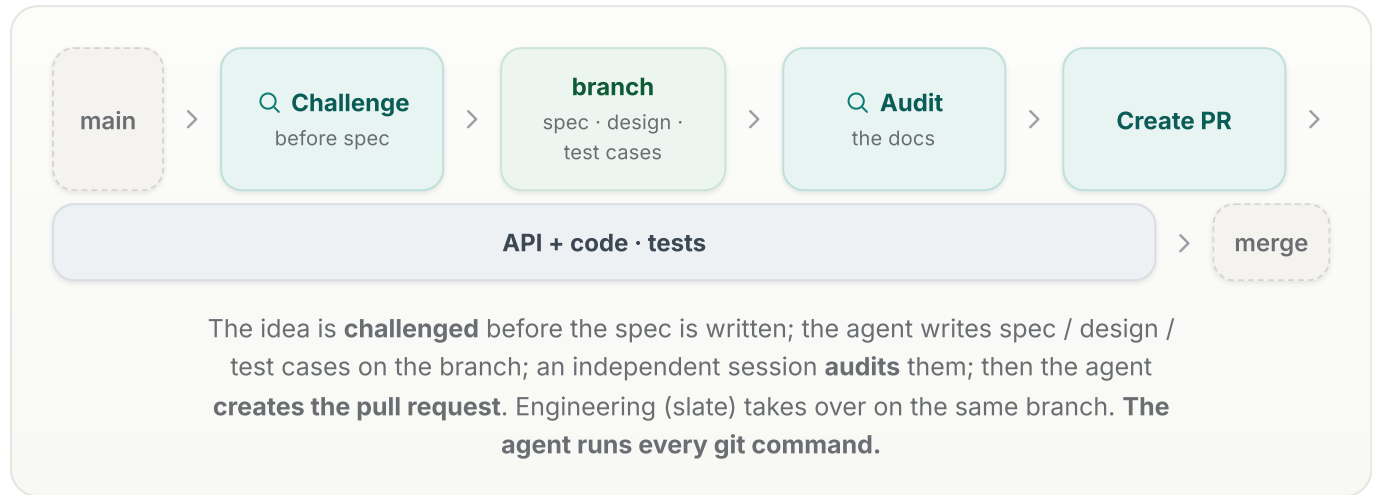
1. Get the project onto your machine and open it in **Claude Code** (the AI agent). An engineer can clone it, confirm the **Pencil** design tool is connected, and confirm the agent can open pull requests (the `gh` CLI).
2. That's it. You **never type a git command or edit a file**. For the two review steps you open a **new** Claude Code session in the same project folder — still no terminal.

🔑 SETUP · RUN ONCE BY AN ENGINEER

```
# One-time, run by an engineer to set the product teammate up:
git clone <repo-url> hris-outsource
cd hris-outsource
# then open the folder in Claude Code, confirm the Pencil (.pen) design tool
# is connected, and confirm the agent can open PRs (the gh CLI is authed).
# After this, the product teammate never uses the terminal — the agent runs all git.
# For the two review steps (02 Challenge, 06 Audit), open a NEW Claude Code session
# in the same folder so the reviewer is independent of the author session.
```

02 One feature → one branch → one pull request

Each idea becomes one feature on its own branch → one pull request. **The agent creates the branch and opens the PR.** Two independent review sessions bracket the build.



03 The seven steps

Seven prompts. Five go to your **author session**; the two 🔍 **teal** ones — Challenge and Audit — go to a **fresh, independent Claude Code session**.

01

Describe

interview

02

🔍 Challenge

fresh session

03

Build spec

FEATURE + PRD

04

Design

screens

05

Test cases

validate TC-#

06

🔍 Audit

fresh session

07

Create PR

opens it for eng

Five author-session steps + two independent fresh-session reviews (teal). The reviews bracket the build: challenge the idea up front, audit the documents before the pull request.

01

Describe your idea — the agent interviews you

You talk; it asks one plain-language question at a time and figures out where it fits.



You do: paste this, fill in the one line, answer its questions. **Copy the summary it gives** — you'll paste it into step 02.

PROMPT · AUTHOR SESSION

I'm on the SWP HRIS product team and I have an idea I want to turn into a spec for engineering.
Be my product analyst and INTERVIEW me – don't expect me to know the format, the IDs, or the structure.

First, quietly read the project context so you understand what already exists: CLAUDE.md, docs/EPICS.md (especially the §8 decisions), docs/design/DESIGN-SYSTEM.md, and the existing FEATURE.md / PRDs for the area my idea touches. Don't dump any of that back at me.

Then interview me to understand what I want:

- Ask ONE question at a time, in plain everyday language – no technical jargon, no IDs, no schemas.
- Whenever you can, give me concrete examples or a few options to pick from, so it's easy to answer.
- Cover, over the conversation: the problem and who has it; which user it's for (super admin / HR / shift leader / agent); where in the app it lives; the step-by-step happy path; the rules and limits; what can go wrong; what each screen should show; what information is involved; and whether this already exists in the current/old system.
- If an answer is vague, gently dig deeper. Don't move on until you genuinely understand.

My idea, in my own words: <DESCRIBE YOUR IDEA HOWEVER YOU LIKE – one sentence or a paragraph>

When you have enough, STOP and show me, in plain language:

1. a short summary of what we're building and for whom;
2. where it fits (brand-new, or a change to something that exists);
3. the list of screens we'll need;
4. anything you still need me to decide.

Then wait for me to say "go". (I'll also copy this summary for an independent challenge step.)

✓ **Done when:** the agent shows a plain-language summary you agree with.

02

Challenge the idea — independent fresh session

A skeptic that hasn't seen your interview pressure-tests the idea before any spec is written.

 **You do: open a NEW Claude Code session** (independent eyes), paste this, and paste the idea + the summary from 01. If the verdict says simplify/reconsider, refine before building.

Q PROMPT · FRESH SESSION

This is a FRESH session – you have NOT seen my earlier conversation, which is the point: I want an independent, skeptical review of an idea BEFORE we spend effort specifying it.

Read the project context: CLAUDE.md, docs/EPICS.md (especially the §8 decisions and the existing epics/features), and the FEATURE.md / PRDs of the area this touches.

Here is the idea and the plan so far:

<PASTE the idea + the plain-language summary the author agent gave you in step 01>

Now CHALLENGE it hard, like a tough product lead – don't be agreeable. Tell me, with specifics:

- Is the problem real and worth solving now? Who actually feels the pain, and how often?
- What's the SIMPLEST version that delivers most of the value? What can we cut?
- Does this DUPLICATE or CONFLICT with something that already exists – an existing feature, a §8 decision, or an always-true rule (invariant)? Cite the specific one.
- What does it BREAK or complicate elsewhere (scheduling, attendance, leave, overtime, payroll, migration)?
- What's MISSING from the idea – an unhandled case, a role, a state, a labour-law constraint?
- What's the riskiest assumption, and how could we cheaply test it?

End with a clear verdict: BUILD AS-IS / BUILD SIMPLER (say how) / RECONSIDER (say why), plus a sharpened one-paragraph problem statement I can take back to refine the spec.

✓ **Done when:** you have a verdict (build / simplify / reconsider) and a sharpened problem statement.

03

Build the spec — the agent writes it, you approve

It makes the branch and writes FEATURE + PRD, then explains it back in plain words.

🔗 **You do:** back in your author session, paste this (mention any change the Challenge produced), then approve or correct the summary.

PROMPT · AUTHOR SESSION

Go. Create the engineering specification from everything I told you (incorporating the points from the independent challenge: <paste the verdict / what changed, or say "no changes">). Do ALL the technical structuring yourself – I shouldn't have to think about formats, IDs, or git.

On your own, without me touching anything:

- create the feature branch and manage it (run the git yourself – I will never use the terminal);
- add the feature to the right area's FEATURE.md – the workflow diagram, the things involved, and the rules that must always hold – matching the exact structure of the features already there;
- write the full PRD in the house format (context, the users, the business rules, the information the feature needs, the acceptance criteria as clear Given/When/Then scenarios, and the edge cases) – allocate all the IDs yourself;
- ground every rule in our locked decisions (EPICS.md §8) and Indonesian labour law where relevant;
- DO NOT touch the API / OpenAPI files or the legacy data-mapping – engineering owns those, not us.

Then show me a PLAIN-LANGUAGE summary: what the feature does, the key rules you wrote, the acceptance criteria in everyday terms, and any assumption you had to make. Ask me to confirm or correct before we design the screens.

✓ **Done when:** the summary matches what you meant.

04

Design the screens — the agent draws them, you look

It draws every screen and every state in the design file and shows you pictures.

🗉 **You do:** paste this, then look at the screenshots and approve the look or point at what to change.

📄 PROMPT · AUTHOR SESSION

Now design the screens for this feature in our design file. Use the Pencil design tools (the design file is special – only you can edit it, through those tools). Follow our design system exactly: reuse the components that already exist, use our standard colours and spacing, keep ALL text in Bahasa Indonesia, and design EVERY state a real screen needs – normal, loading, empty, error, and no-access – so nothing is left undesigned. Start by copying an existing similar screen, then adapt it.

When you're done, show me screenshots of each screen and each state, and tell me in plain language what the user does on each one. Keep a note of the screen ids so engineering can find them. Then ask me to approve the look or tell you what to change.

✓ **Done when:** the screenshots look right to you.

05

Write & validate the test cases — the agent proves it's testable

From the finished docs it generates `TC-#` and checks they cover every rule, scenario, edge case, and screen state.

✓ **You do:** paste this **after the spec and screens are settled**. If it flags a spec gap, fix it first.

PROMPT · AUTHOR SESSION

Now that the spec and the screens are settled, generate the TEST CASES from the documentation – this is how we prove the feature is fully testable before engineering builds it.

Read the finished FEATURE.md and the PRD (especially the business rules, the acceptance criteria, and the edge cases). Then:

- write a test-case document next to the PRD at docs/epics/E<#>-<name>/prds/<slug>.test-cases.md, with a numbered list of test cases (TC-#). For each: a clear title; the type (happy path / edge / negative); the preconditions; the steps in plain language; the expected result; the screen + state it exercises; and a trace back to what it checks (which business rule BR-#, which acceptance scenario, which C-#).
- cover EVERYTHING: every business rule, every acceptance-criteria scenario, every edge case, and every screen state must have at least one test case.

Then VALIDATE the test cases against the documentation and tell me, in plain language:

- a coverage table – each BR-# / acceptance scenario / C-# and the test case(s) that cover it;
- anything NOT yet covered (a gap), and whether the gap is a missing test or a hole in the spec/design;
- any test case you can't decide from the docs (the rule is ambiguous) – flag it so we settle it.

If you find gaps in the spec or design, list them and ask me how to resolve them before we finalise.

✓ **Done when:** the coverage table has no gaps.

Audit completeness + cohesion — independent fresh session

06

A strict reviewer that hasn't seen the build checks the docs are complete and fit the existing system.

✓ **You do: open a NEW Claude Code session** in the same project folder (the docs are on the branch). Paste this; fix whatever it lists, then re-run if needed.

PROMPT · FRESH SESSION

This is a FRESH session – you have NOT seen how these documents were written, which is the point: I want an INDEPENDENT audit of the documentation for feature F<#.>, for COMPLETENESS and for COHESION with the existing system. Be a strict reviewer; assume nothing is right until you've checked it against the files.

Read: the new FEATURE.md entry, the PRD (prds/<slug>.md), the test cases (prds/<slug>.test-cases.md), AND the things they must fit into – docs/EPICS.md §8 (the authoritative decision log), the sibling features in the same epic, the invariants (INV-#), docs/api/CONVENTIONS.md, and docs/design/DESIGN-SYSTEM.md.

Produce a report with PASS/FAIL per item and, for each FAIL, the exact fix:

COMPLETENESS

- PRD has all sections in the house order; every business rule (BR-#) is testable and tied to an acceptance-criteria scenario; every edge case (C-#) is captured.
- Every BR-# / acceptance scenario / C-# / screen state has at least one test case (TC-#) – no gap.
- Every screen has all its states designed (no dead-flow); frame ids are recorded.
- Nothing is left as an unresolved "TBD" that should have been decided.

COHESION WITH THE EXISTING SYSTEM

- Nothing contradicts EPICS.md §8 (quote the exact decision it would violate, if any).
- No clash with an existing feature, an invariant (INV-#), or the role/permission model; IDs don't collide.
- Consistent with the API conventions and the design system (tokens, no-dead-flow, Bahasa, Asia/Jakarta).
- Integration points with other epics (scheduling, attendance, leave, overtime, payroll, migration) are named and consistent – flag anything this feature affects but doesn't mention.
- Labour-law facts (PKWT/PKWTT, alih-daya) are correct.

End with one line: READY FOR ENGINEERING, or a numbered list of what must be fixed first.

✓ **Done when:** the audit says **READY FOR ENGINEERING**.

07

Create the pull request — the agent opens it for engineering

It does a final self-check, commits the spec + design + test cases, and **creates the pull request**.

 **You do:** paste this. The agent self-checks, commits, pushes, and creates the pull request, then gives you the link.

PROMPT · AUTHOR SESSION

Send this to engineering by CREATING the pull request for our branch. First, do a final self-check: the rules, the acceptance criteria, the edge cases, every screen state, AND the test cases are all present, fully cover each other, the independent audit passed, and nothing contradicts EPICS.md §8. Fix any gap.

Then:

- commit the spec documents, the design, and the test cases with clear messages;
- PUSH the branch and CREATE the pull request (open it as a Draft) titled with the feature name. Write the description in plain language PLUS the technical detail engineers need: the problem, the rules, the acceptance criteria, the test-case coverage (how many TC-#, and what they cover), the screen ids, and a note that the independent challenge + audit passed;
- state clearly in the PR that engineering owns the API contract, the data mapping, and the code, and that they implement our TC-# test cases as automated tests.

Finally, give me the pull-request link and one short paragraph: what you handed over and what happens next.

✓ **Done when:** the agent gives you the pull-request link. Engineering takes it from here.

04 "Ready for engineering" — the checks behind the hand-off

Two layers make a feature ready: the **independent audit** (Step 06, the real gate) and the author agent's **own self-check** (Step 07). Both confirm:

- ✓ The idea survived the **Challenge** (02) — worth building, scoped right, no duplication/conflict.
- ✓ The idea is placed in the right area (new feature / change / decision).
- ✓ **FEATURE.md** has the feature: its workflow, the things involved, and the always-true rules.
- ✓ The **PRD** is complete: problem, users, business rules, data needed, **acceptance criteria** (Given/When/Then), and edge cases.
- ✓ Every screen exists with **all** states (normal, loading, empty, error, no-access) — no dead-flow.
- ✓ **Test cases** (**TC-#**) exist and are **validated**: full coverage, no gaps.
- ✓ **Cohesion** confirmed by the independent audit: nothing contradicts **EPICS.md §8**, invariants, existing features, the API conventions, or the design system; integration points named.
- ✓ Any rule/policy that changed is a **dated decision**; copy is Bahasa; dates **Asia/Jakarta**; labour-law facts right.

🔧 THEN ENGINEERING ADDS — NOT YOUR CONCERN

The API contract, the legacy data-mapping, the typed client, the code, and the **automated tests that implement your TC-#**.

05 Engineering pickup

Engineering takes the pull request and implements on the **same branch** — owning the API contract and the data-mapping, and **turning the product's TC-# test cases into automated tests**.

PROMPT · ENGINEERING — IMPLEMENT FROM SETTLED DOCS

Implement feature F<#> from its settled documentation. CODE task — follow docs/eng/ENGINEERING.md and docs/eng/WEB-STACK.md exactly.

1. Read the PRD (docs/epics/E<#>-<name>/prds/<slug>.md) — the Gherkin AC is your acceptance contract — the FEATURE.md (entities, INV-#), and the product test cases (prds/<slug>.test-cases.md, the TC-#).
2. AUTHOR THE API CONTRACT the product side deliberately left to you: write/extend docs/api/E<#>-<name>/openapi.yaml (OpenAPI 3.1) from the PRD — paths/verbs, schemas, and:
 - x-rbac: self.* is a BASELINE (any authenticated employee, scope:self); elevations are lead / hr_admin / super_admin; shift_leader is DERIVED from the E3 assignment, never stored.
 - x-design-screens linking each operation to the .pen frame ids recorded in the PRD.
 - errors mapped to BR-#/INV-#; CURSOR pagination only; Idempotency-Key on every write.Inherit docs/api/CONVENTIONS.md (don't restate it; extend CONVENTIONS if a rule is missing).
3. If the feature touches persisted legacy data, write/update docs/epics/E<#>-<name>/DATA-MAPPING.md (legacy MySQL lumen_swp → Postgres: employee_contracts, companies.role=2 = client, users.id vs employees.id, DBEncryption payroll columns) and note any new E9 transform.
4. Regenerate the typed client (Orval). NEVER hand-edit generated files (E2); drift fails CI.
5. Build screens from the cited .pen frames via the Pencil MCP (G0 — never from assumptions); assemble from packages/ui + tokens; implement EVERY state variant (B2 no dead-flow); errors via classifyError (B1); idempotency keys on mutations (C3).
6. IMPLEMENT THE PRODUCT'S TEST CASES: turn each TC-# into an automated test — critical happy-path TC-# as Playwright E2E, rule/edge TC-# (BR-#/C-#) as Vitest against MSW (F1/F2) — and keep the TC-# id in the test name so coverage traces straight back to the product test-case doc. Every TC-# must have a test; flag any TC-# you cannot automate and why.
7. Cite F#/BR-#/C-#/TC-# + the .pen frame id in each commit. Push to the same branch; flip the PR to Ready when pnpm typecheck && pnpm lint && pnpm test are green and every TC-# is covered.

✓ THE ACCEPTANCE CRITERIA + TEST CASES ARE THE CONTRACT

If something is ambiguous during the build, the fix is to update the PRD/AC/test case (a small doc commit), **not** to decide it in code.

What gets written, and by whom



THE AGENT WRITES — FROM YOUR INTERVIEW

FEATURE.md

workflow · entities · always-true rules

prds/<slug>.md

rules · acceptance criteria · edge cases

<slug>.test-cases.md

TC-# · traces to BR-#/AC/C-#

brainstorm.pen

screen frames (all states)

SCREEN-GENERATION-PLAN.md

build tracker



ENGINEERING WRITES

api/E#/openapi.yaml

from the PRD · x-rbac · x-design-screens

DATA-MAPPING.md

legacy lumen_swp → Postgres

automated tests

Playwright / Vitest — implement each TC-#

E9 migration

transform-and-load at cutover

EPICS.md §8 is the authority every layer reconciles to. Engineering derives the API contract + data-mapping from the PRD, and automates each TC-#.

06

Roles & responsibilities

STEP	PRODUCT	ENG	DRIVEN BY
01 · Describe & interview	R/A	—	author session (asks you)
02 · Challenge the idea	R/A	—	🔍 independent fresh session
03 · Build the spec	A approve	C	author session (writes it)
04 · Design the screens	A approve	C	author session (draws it)
05 · Write & validate test cases	A approve	C	author session
06 · Audit completeness + cohesion	R/A	C	🛡 independent fresh session
07 · Create the pull request	A approve	C	author session (opens it)
Automate TC-# · API · data · code	—	R/A	engineering
Review + merge	C	R/A	engineering

R responsible · **A** accountable · **C** consulted. The agent is the tool both sides drive; the human named **A** owns the outcome.

07 Quick reference — the prompt library

STEP	SESSION	YOU PASTE...	THE AGENT PRODUCES
01	author	"Describe your idea"	an interview → a plain-language plan
02	 fresh	"Challenge the idea"	a skeptical verdict — build / simplify / reconsider
03	author	"Build the spec"	branch + <code>FEATURE.md</code> + PRD
04	author	"Design the screens"	the <code>.pen</code> screens + screenshots
05	author	"Write & validate test cases"	<code><slug>.test-cases.md</code> + coverage table
06	 fresh	"Audit completeness + cohesion"	PASS/FAIL → READY FOR ENGINEERING
07	author	"Create the pull request"	self-check + commits + the PR (link)
	eng	"Implement from settled docs"	API + data-mapping + code + automated tests

TIPS

Run 02 Challenge and 06 Audit in a **separate Claude Code session** — independence is what makes them useful. Do 05 test cases and 06 audit **only after** the spec and screens are settled. If the agent asks something you don't know, *"I'm not sure — what do you recommend?"* is a fine answer.

Source of truth: `docs/workflow/PRODUCT-ENG-WORKFLOW.md`. This page and the PDF are renders of it — edit the Markdown, re-render.